

Pw3c DISTRIBUIDORA

Sistema de Gestión de Distribución

Manual Técnico

Versión 1.0 • Junio 2026

Laravel 9 • PHP 8 • MySQL • JavaScript Vanilla

© 2026 Pw3c Distribuidora. Todos los derechos reservados.

Índice General

1. Introducción
2. Arquitectura del Sistema
3. Requisitos del Sistema
4. Instalación y Configuración
5. Estructura del Proyecto
6. Base de Datos
7. API REST
8. Sistema de Autenticación
9. Sistema de Roles y Permisos (RBAC)
10. Frontend
11. Controladores
12. Mantenimiento

Capítulo 1: Introducción

1.1 Propósito del Manual

Este manual técnico describe la arquitectura, instalación, configuración y mantenimiento del sistema Pw3c Distribuidora. Está dirigido a desarrolladores, administradores de sistemas y personal técnico encargado del despliegue y soporte de la aplicación.

1.2 Descripción del Sistema

Pw3c Distribuidora es un sistema de gestión de distribución farmacéutica que implementa un modelo cliente-servidor con una API RESTful en Laravel 9 y un frontend independiente en JavaScript vanilla. El sistema utiliza autenticación mediante tokens Sanctum y un sistema de control de acceso basado en roles (RBAC) con 11 módulos y 13 acciones.

1.3 Tecnologías Utilizadas

Tecnología	Versión	Propósito
PHP	^8.0	Lenguaje de programación del backend
Laravel	^9.0	Framework de desarrollo web
Laravel Sanctum	^2.14	Autenticación por tokens
MySQL / MariaDB	5.7+ / 10.3+	Base de datos relacional
Composer	2.x	Gestor de dependencias PHP
JavaScript (Vanilla)	ES6+	Frontend del sistema
jQuery	3.7.1	Manipulación del DOM
DataTables	1.13.6	Tablas dinámicas
Chart.js	4.4.1	Gráficos interactivos
SweetAlert2	11.x	Cuadros de diálogo
Font Awesome	6.5.1	Iconografía

Capítulo 2: Arquitectura del Sistema

2.1 Arquitectura General

El sistema sigue una arquitectura de dos capas desacopladas:

- Backend API (Laravel 9): Servidor de aplicaciones que expone una API RESTful en el puerto 8000. Gestiona la lógica de negocio, base de datos, autenticación y autorización.
- Frontend (Vanilla JS): Aplicación web independiente servida por Live Server en el puerto 5500. Consume la API mediante fetch() y gestiona el estado de sesión en localStorage.

2.2 Diagrama de Flujo de Datos

El flujo de datos sigue este patrón:

1. El usuario interactúa con la interfaz HTML/CSS/JS
2. JavaScript realiza peticiones fetch() a la API de Laravel
3. Laravel procesa la solicitud, aplica middleware (auth:sanctum, permission)
4. El controlador ejecuta la lógica de negocio y consulta la base de datos
5. La respuesta JSON viaja de vuelta al frontend
6. JavaScript actualiza el DOM con los datos recibidos

2.3 Separación Frontend/Backend

El frontend y el backend son proyectos independientes:

- Backend: carpeta raíz del proyecto Laravel
- Frontend: subdirectorio frontenda_distribuidora/
- Comunicación: únicamente vía HTTP (API REST)
- Autenticación: tokens Bearer almacenados en localStorage

La URL base de la API se define en frontenda_distribuidora/js/config.js:

```
const API_URL = 'http://127.0.0.1:8000/api';
```

Capítulo 3: Requisitos del Sistema

3.1 Requisitos de Software

Componente	Requisito Mínimo
PHP	8.0 o superior
Compositor	2.x
Base de Datos	MySQL 5.7+ o MariaDB 10.3+
Servidor Web	Apache / Nginx / Laravel Artisan
Node.js	16+ (para Laravel Mix, opcional)
Navegador	Chrome 90+, Firefox 88+, Edge 90+
Extensión PHP	BCMath, CType, Fileinfo, JSON, Mbstring, OpenSSL, PDO, Tokenizer, XML, cURL

3.2 Estructura de Archivos de Configuración

- .env: configuración de base de datos, APP_URL, etc.
- config/database.php: conexión a MySQL
- config/sanctum.php: configuración de tokens
- config/cors.php: configuración CORS para frontend
- config/filesystems.php: disco público para evidencias

Capítulo 4: Instalación y Configuración

4.1 Instalación del Backend

7. Clonar el repositorio del proyecto
8. Ejecutar "composer install" para instalar dependencias PHP
9. Copiar .env.example a .env y configurar:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=pw3c_distribuidora
DB_USERNAME=root
DB_PASSWORD=
```

10. Generar clave de aplicación: php artisan key:generate
11. Ejecutar migraciones: php artisan migrate
12. Poblar base de datos: php artisan db:seed
13. Crear enlace simbólico para archivos: php artisan storage:link
14. Iniciar servidor: php artisan serve

4.2 Instalación del Frontend

El frontend es una aplicación web estática que se sirve con cualquier servidor HTTP:

15. Opción A: Abrir frontenda_distribuidora/index.html directamente
16. Opción B: Usar Live Server de VS Code sobre la carpeta frontenda_distribuidora/
17. Opción C: Configurar un virtual host de Apache/Nginx que apunte a frontenda_distribuidora/

4.3 Configuración CORS

Para permitir peticiones desde el frontend (puerto 5500) al backend (puerto 8000), configurar config/cors.php:

```
'allowed_origins' => ['http://127.0.0.1:5500', 'http://localhost:5500'],
```

4.4 Verificación de la Instalación

Para verificar que la instalación fue exitosa:

18. Acceder a http://127.0.0.1:8000/api/auth/user con GET (debe devolver 401 sin token)
19. Iniciar sesión vía POST a /api/auth/login con email=admin@distribuidora.com y password=123456
20. Usar el token recibido para autenticar otras peticiones
21. Abrir el frontend en http://127.0.0.1:5500 y verificar que carga correctamente

Capítulo 5: Estructura del Proyecto

5.1 Backend (Laravel)

Directorio	Contenido
app/Http/Controllers/	15 controladores organizados por dominio
app/Http/Middleware/	Middleware CheckPermission (RBAC)
app/Models/	36 modelos Eloquent organizados por dominio
database/migrations/	42 migraciones de base de datos
database/seeds/	CatalogoSeeder y DemoDataSeeder
routes/api.php	Todas las rutas de la API (~90 endpoints)
config/	Configuración de Laravel (cors, sanctum, filesystems)
public/	Punto de entrada del servidor web
storage/app/public/	Archivos subidos (evidencias)

5.2 Frontend (frontenda_distribuidora/)

Directorio/Archivo	Contenido
index.html	Página de inicio de sesión / registro
dashboard.html	Dashboard principal con estadísticas y gráficos
pages/	10 páginas HTML (usuarios, roles, farmacias, etc.)
js/config.js	Configuración global (API_URL, apiFetch, hasPermission)
js/auth.js	Autenticación (login, register, logout, checkAuth)
js/layout.js	Barra lateral dinámica con filtro por permisos
js/error-handler.js	Manejo global de errores
js/modules/	12 módulos JS (uno por cada funcionalidad)
css/style.css	Estilos principales del sistema
css/login.css	Estilos de la página de login

5.3 Organización de Modelos por Dominio

Dominio (Namespace)	Modelos
Seguridad	Usuario, Rol, Modulo, Accion, Permiso, EstadoUsuario, Auditoria, SesionUsuario, TablaSistema
Farmacia	Farmacia, ContactoFarmacia, Cargo
Pedido	Pedido, EstadoPedido, HistorialEstadoPedido
Repartidor	Repartidor, EstadoRepartidor, ExtensionCI, Licencia, HistorialEstadoRepartidor
Vehiculo	Vehiculo, Marca, Modelo, Capacidad, EstadoVehiculo, HistorialEstadoVehiculo
Logistica	Ruta, RutaParada, ControlRuta
Despacho	Despacho, EstadoDespacho, HistorialEstadoDespacho
Evidencia	Incidencia, TipoIncidencia, EvidenciaEntrega, TipoEvidencia

Capítulo 6: Base de Datos

6.1 Resumen

La base de datos consta de 42 migraciones que crean tablas para los dominios de seguridad, farmacias, pedidos, repartidores, vehículos, rutas, despachos y evidencias. Todas las claves foráneas están correctamente definidas con restricciones de integridad referencial.

6.2 Tablas de Seguridad (9)

Tabla	PK	FK	Descripción
estados_usuario	id_estado_usuario	—	Catálogo: Activo, Bloqueado, Suspendido
roles	id_rol	—	Roles del sistema (Admin, Supervisor, Operador, Repartidor)
modulos	id_modulo	—	Módulos del sistema (11 registros)
acciones	id_accion	—	Acciones del sistema (13 registros)
permisos	id_permiso	id_modulo, id_accion	Combinaciones módulo-acción (59 permisos)
rol_permiso	—	id_rol, id_permiso	Asignación de permisos a roles (tabla pivote)
usuarios	id_usuario	id_estado_usuario	Usuarios del sistema
usuario_rols	—	id_usuario, id_rol	Roles asignados a usuarios (tabla pivote)
auditorias	id_auditoria	id_usuario	Registro de auditoría (no implementado)
sesiones_usuario	id_sesion	id_usuario	Sesiones de usuario (no implementado)

6.3 Tablas de Dominio

Farmacias:

Tabla	PK	FK
farmacias	id_farmacia	id_cargo (opcional)
contactos_farmacia	id_contacto	id_farmacia, id_cargo
cargos	id_cargo	—

Pedidos:

Tabla	PK	FK
pedidos	id_pedido	id_farmacia, id_usuario, id_estado_pedido
historial_estado_pedido	id_historial	id_pedido, id_estado_pedido, id_usuario

Repartidores:

Tabla	PK	FK
repartidores	id_repartidor	id_usuario, id_estado_repartidor, id_extension_ci, id_licencia
historial_estado_repartidor	id_historial	id_repartidor, id_estado_repartidor

Vehículos:

Tabla	PK	FK
vehiculos	id_vehiculo	id_marca, id_modelo, id_capacidad, id_estado_vehiculo
marcas	id_marca	—
modelos	id_modelo	id_marca
capacidades	id_capacidad	—
historial_estado_vehiculo	id_historial	id_vehiculo, id_estado_vehiculo

Rutas y Despachos:

Tabla	PK	FK
rutas	id_ruta	—
ruta_paradas	id_parada	id_ruta, id_farmacia
control_rutas	id_control_ruta	id_ruta, id_repartidor, id_vehiculo
despachos	id_despacho	id_pedido, id_parada, id_control_ruta, id_estado_despacho
historial_estado_despacho	id_historial	id_despacho, id_estado_despacho

Evidencias:

Tabla	PK	FK
incidencias	id_incidencia	id_despacho, id_tipo_incidencia
evidencias_entrega	id_evidencia	id_despacho, id_tipo_evidencia
tipos_incidencia	id_tipo_incidencia	—
tipos_evidencia	id_tipo_evidencia	—

6.4 Estados por Dominio

Tabla de Estados	Valores
estados_usuario	Activo, Bloqueado, Suspendido
estados_pedido	Pendiente, Aprobado, En Preparación, Despachado, Entregado, Cancelado
estados_repartidor	Disponible, En Ruta, Inactivo
estados_vehiculo	Operativo, En Mantenimiento, Fuera de Servicio
estados_despacho	Pendiente, En Camino, Entregado, Fallido

6.5 Migraciones

Las 42 migraciones se ejecutan en orden cronológico: primero los catálogos (01_*), luego las entidades principales (02_*), luego las tablas de relación (03_*), y finalmente los históricos y evidencias (04_*).

Capítulo 7: API REST

7.1 Convenciones

Todas las respuestas de la API siguen el formato unificado:

```
{
  "success": true/false,
  "data": { ... } | [ ... ],
  "message": "Mensaje opcional"
}
```

7.2 Endpoints de Autenticación

Método	URI	Controlador	Middleware
POST	/auth/login	AuthController@login	Ninguno
POST	/auth/register	AuthController@register	Ninguno
POST	/auth/logout	AuthController@logout	auth:sanctum
GET	/auth/user	AuthController@user	auth:sanctum

7.3 Endpoints CRUD por Módulo

Cada módulo expone endpoints RESTful con el patrón apiResource:

Método	URI	Función	Permiso
GET	/recurso	index (listar)	Módulo:listar
GET	/recurso/{id}	show (ver)	Módulo:listar
POST	/recurso	store (crear)	Módulo:crear
PUT/PATCH	/recurso/{id}	update (editar)	Módulo:editar
DELETE	/recurso/{id}	destroy (eliminar)	Módulo:eliminar

Recursos disponibles: usuarios, roles, farmacias, pedidos, repartidores, vehiculos, rutas, controles-ruta, despachos.

7.4 Endpoints de Catálogos

Método	URI	Descripción	Middleware
GET	/catalogos/{nombre}	Listar registros del catálogo	auth:sanctum
POST	/catalogos/{catalogo}	Crear registro	permission:Usuarios
PUT	/catalogos/{catalogo}/{id}	Actualizar registro	permission:Usuarios
DELETE	/catalogos/{catalogo}/{id}	Eliminar registro	permission:Usuarios

Catálogos disponibles (17): estados-usuario, roles, modulos, acciones, tablas-sistema, estados-pedido, estados-repartidor, estados-vehiculo, estados-despacho, extensiones-ci, licencias, marcas, modelos, capacidades, tipos-incidencia, tipos-evidencia, cargos.

7.5 Endpoints de Reportes

Método	URI	Función	Permiso
GET	/reportes/resumen	resumen	Reportes:acceder
GET	/reportes/pedidos-por-estado	pedidosPorEstado	Reportes:listar
GET	/reportes/despachos-por-estado	despachosPorEstado	Reportes:listar
GET	/reportes/pedidos-por-dia	pedidosPorDia	Reportes:listar
GET	/reportes/repartidores-por-estado	repartidoresPorEstado	Reportes:listar
GET	/reportes/vehiculos-por-	vehiculosPorEstado	Reportes:listar

	estado		
GET	/reportes/incidencias-por-tipo	incidenciasPorTipo	Reportes:listar

7.6 Endpoints Anidados

El sistema incluye rutas anidadas para recursos dependientes:

URI	Descripción
/farmacias/{id}/contactos	CRUD de contactos de una farmacia
/rutas/{id}/paradas	CRUD de paradas de una ruta
/despachos/{id}/incidencias	CRUD de incidencias de un despacho
/despachos/{id}/evidencias	CRUD de evidencias de un despacho

Capítulo 8: Sistema de Autenticación

8.1 Tecnología

La autenticación se implementa con Laravel Sanctum, que proporciona un sistema de tokens API simples. Cada token es un hash SHA-256 almacenado en la tabla `personal_access_tokens`.

8.2 Flujo de Autenticación

22. El usuario envía credenciales (email + password) a `POST /api/auth/login`
23. `AuthController@login` verifica las credenciales con `Hash::check()`
24. Si son válidas, genera un token con `$user->createToken('api-token')`
25. El token se devuelve en la respuesta JSON junto con datos del usuario y permisos
26. El frontend almacena el token en `localStorage`
27. Todas las peticiones posteriores incluyen el header: `Authorization: Bearer {token}`

8.3 Implementación en Frontend

El módulo `auth.js` gestiona todo el ciclo de autenticación:

- `login()`: envía credenciales, guarda token + usuario + permisos en `localStorage`
- `register()`: crea usuario, asigna rol Repartidor por defecto (`id=4`), guarda sesión
- `logout()`: elimina token en backend, limpia `localStorage`, redirige a login
- `checkAuth()`: verifica que exista un token válido en `localStorage`
- `loadUser()`: refresca datos del usuario y permisos desde la API

8.4 `apiFetch (config.js)`

La función `apiFetch()` es el único punto de acceso a la API desde el frontend:

- Inyecta automáticamente el header `Authorization: Bearer {token}`
- Maneja errores 401 (sesión expirada) limpiando `localStorage` y redirigiendo
- Convierte automáticamente objetos JavaScript a JSON en el body
- Soporta `FormData` para subida de archivos
- Registra logs de depuración en consola para cada petición

Capítulo 9: Sistema de Roles y Permisos (RBAC)

9.1 Modelo de Datos

El RBAC se implementa con 4 tablas principales:

- módulos: 11 registros (Dashboard, Reportes, Usuarios, Roles, Farmacias, Pedidos, Repartidores, Vehículos, Rutas, Control Rutas, Despachos)
- acciones: 13 registros (acceder, listar, crear, editar, eliminar, asignar-roles, asignar-permisos, gestionar-contactos, cambiar-estado, gestionar-paradas, registrar-llegada, gestionar-incidencias, gestionar-evidencias)
- permisos: 59 combinaciones módulo-acción
- rol_permiso: tabla pivote que asigna permisos a roles

9.2 Middleware CheckPermission

El middleware se registra como permission en app/Http/Kernel.php y acepta dos parámetros: el módulo (obligatorio) y la acción (opcional).

Maapeo automático de métodos HTTP a acciones:

Método Controlador	HTTP	Acción
index	GET	listar
show	GET {id}	listar
store	POST	crear
update	PUT/PATCH	editar
destroy	DELETE	eliminar

9.3 Método hasPermission

El modelo Usuario implementa hasPermission(\$module, \$action) que:

28. Recupera todos los roles del usuario
29. Itera sobre los permisos de cada rol
30. Compara el nombre del módulo y la acción contra los valores recibidos
31. Retorna true si encuentra al menos una coincidencia

9.4 Roles Predefinidos y sus Permisos

Rol	# Permisos	Módulos sin Acceso
Administrador	59 (todos)	Ninguno
Supervisor	40	Usuarios, Roles
Operador	25	Usuarios, Roles, escritura en Farmacias, Repartidores, Vehículos, Rutas
Repartidor	8	Solo Dashboard, Rutas (ver), Control Rutas (ver+llegada), Despachos (ver)

Capítulo 10: Frontend

10.1 Estructura General

El frontend es una aplicación de página única (SPA-like) construida con JavaScript vanilla. Cada página HTML carga su módulo JS correspondiente que se encarga de toda la lógica de presentación.

10.2 Patrón de Cada Página

Cada módulo JS sigue el mismo patrón:

```
document.addEventListener('DOMContentLoaded', async function () {
  if (!checkAuth()) return;
  if (!hasPermission('Module', 'acceder')) {
    window.location.href = '../dashboard.html'; return;
  }
  await loadCatalogos();
  renderPage();
});
```

10.3 Archivos Clave

- config.js: API_URL, apiFetch(), hasPermission(), getStatusBadge(), formatDate(), reloadDataTable()
- auth.js: login(), register(), logout(), checkAuth(), loadUser(), getUser()
- layout.js: sidebar dinámico con filtro por permisos del usuario

10.4 Bibliotecas Externas

Biblioteca	Uso
jQuery 3.7.1	Manipulación del DOM, eventos
DataTables 1.13.6	Tablas con ordenamiento, búsqueda y paginación
Chart.js 4.4.1	Gráficos de dona y línea en Dashboard y Reportes
SweetAlert2 11	Cuadros de diálogo modernos (confirmaciones, formularios)
FontAwesome 6.5.1	Iconos en toda la interfaz

10.5 Función hasPermission en el Frontend

Los permisos se cargan al iniciar sesión y se almacenan en localStorage. La función hasPermission(module, action) verifica contra este arreglo:

```
function hasPermission(modulo, accion) {
  return userPermissions.some(
    p => p.modulo === modulo && p.accion === accion
  );
}
```

Capítulo 11: Controladores

11.1 ApiController (Base)

Clase base que proporciona métodos para respuestas JSON uniformes:

```
jsonResponse($data, $message = null, $status = 200)
errorResponse($message, $status = 400)
```

11.2 Listado de Controladores

Controlador	Namespace	Métodos Principales
AuthController	App\Http\Controllers	login, register, logout, user
CatalogoController	App\Http\Controllers	17 métodos GET + store, update, destroy genéricos
UsuarioController	Seguridad	CRUD + roles, assignRoles
RolController	Seguridad	CRUD + permisos, assignPermisos
FarmaciaController	Farmacia	CRUD
ContactoFarmaciaController	Farmacia	CRUD anidado por farmacia
PedidoController	Pedido	CRUD + cambiarEstado
RepartidorController	Repartidor	CRUD + cambiarEstado
VehiculoController	Vehiculo	CRUD + cambiarEstado
RutaController	Logistica	CRUD + paradas, storeParada, updateParada, destroyParada
ControlRutaController	Logistica	CRUD + registrarLlegada
DespachoController	Despacho	CRUD + cambiarEstado
IncidenciaController	Despacho	CRUD anidado por despacho
EvidenciaController	Despacho	CRUD anidado + subida de archivos
ReporteController	Reportes	resumen, 6 reportes gráficos

11.3 CatalogoController

Este controlador maneja los 17 catálogos del sistema de forma genérica. Cada método GET retorna el modelo correspondiente con sus relaciones. Los métodos store, update y destroy reciben el nombre del catálogo como parámetro de ruta y usan un mapeo interno para determinar el modelo y sus reglas de validación.

Capítulo 12: Mantenimiento

12.1 Comandos Artisan Útiles

Comando	Descripción
php artisan migrate	Ejecuta migraciones pendientes
php artisan migrate:fresh	Borra todas las tablas y ejecuta migraciones desde cero
php artisan db:seed	Ejecuta los seeders (catálogos + datos demo)
php artisan migrate:fresh --seed	Reconstruye toda la BD con datos de prueba
php artisan storage:link	Crea enlace simbólico para archivos subidos
php artisan serve	Inicia servidor de desarrollo
php artisan route:list	Lista todas las rutas registradas
php artisan cache:clear	Limpia la caché de la aplicación
php artisan config:cache	Cachea la configuración

12.2 Respaldo de Base de Datos

Para respaldar la base de datos:

```
mysqldump -u root pw3c_distribuidora > respaldo_$(date +%Y%m%d).sql
```

Para restaurar:

```
mysql -u root pw3c_distribuidora < respaldo_20260601.sql
```

12.3 Solución de Problemas Comunes

Problema	Solución
Token no válido al iniciar sesión	Limpiar localStorage del navegador (F12 > Application > Clear storage)
Error CORS al conectar frontend	Verificar config/cors.php y reiniciar servidor
Error 500 en peticiones API	Revisar storage/logs/laravel.log para detalles del error
Imágenes de evidencias no cargan	Ejecutar php artisan storage:link para crear el enlace simbólico
Permisos no se actualizan	Cerrar sesión y volver a iniciar para recargar permisos
Error "Class not found" al correr seeders	Ejecutar composer dump-autoload
Puerto 8000 ocupado	Usar php artisan serve --port=8001 y actualizar API_URL en config.js

12.4 Logs del Sistema

Laravel registra todos los errores en storage/logs/laravel.log. El frontend registra logs de depuración en la consola del navegador (F12) para cada petición API.

12.5 Actualización del Sistema

32. Respaldo base de datos actual
33. Actualizar código fuente (git pull)
34. Ejecutar composer install para actualizar dependencias
35. Ejecutar php artisan migrate para aplicar nuevas migraciones
36. Verificar que todo funcione correctamente